

Ingeniería Web Avanzada

Repaso de Conceptos

Sandra Cano

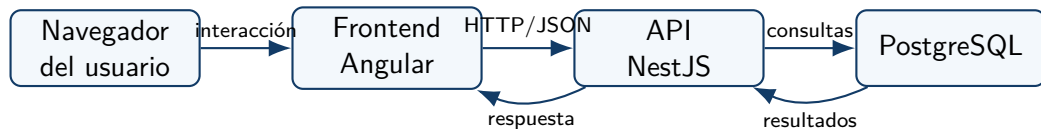
Pontificia Universidad Católica de Valparaíso
OII436-1

Segundo semestre de 2026

Diagnóstico: preguntas conceptuales

- 1 ¿Qué diferencia existe entre un **cliente** y un **servidor**?
- 2 ¿Qué responsabilidades deberían pertenecer al frontend, al backend y a la base de datos?
- 3 ¿Qué representa una petición HTTP?
- 4 ¿Cuándo utilizaría GET, POST, PUT, PATCH y DELETE?
- 5 ¿Qué significan los códigos 200, 201, 400, 401, 403, 404 y 500?
- 6 ¿Qué problema aparece cuando un componente conoce demasiados detalles de otro?
- 7 ¿Cómo comprobaría que una modificación no rompió el sistema?
- 8 ¿Qué información necesitaría para diagnosticar una falla en producción?

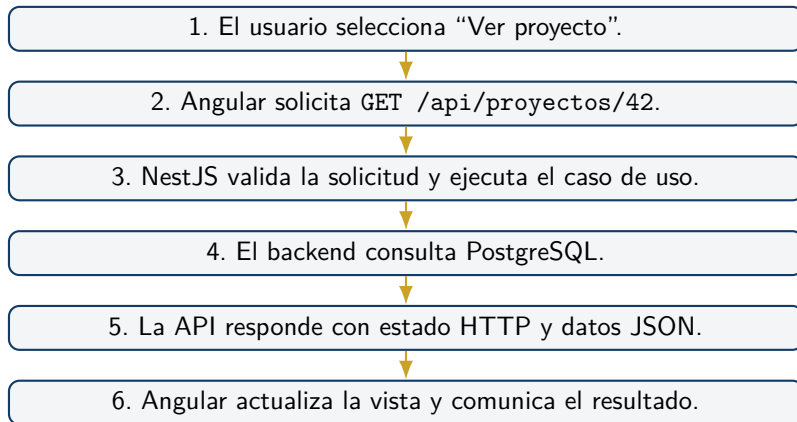
Diagnóstico: lectura de una arquitectura



Pregunta

Identifique una responsabilidad principal y un posible error de diseño en cada componente.

Flujo básico de una interacción web



Responsabilidades: ¿dónde debería vivir cada decisión?

Decisión	Ubicación recomendada	Justificación
Mostrar u ocultar un panel	Frontend	Es una decisión de presentación.
Validar reglas del negocio	Backend/dominio	Debe cumplirse para cualquier cliente.
Garantizar unicidad de correo	Backend y base de datos	Requiere validación y restricción persistente.
Ordenar visualmente una tabla	Frontend	Depende de la experiencia de usuario.
Autorizar acceso a un recurso	Backend	No debe depender de controles visuales.
Mantener integridad referencial	Base de datos	Protege la consistencia persistente.

Error frecuente

Confundir validación de interfaz con seguridad. Ocultar un botón no impide una petición directa a la API.

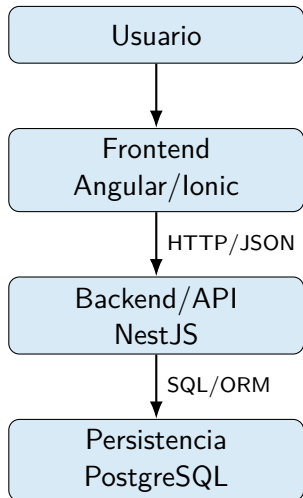
Arquitectura, patrón y tecnología

Concepto	Ejemplos
Arquitectura o estilo	Cliente-servidor, capas, monolito modular, microservicios, orientada a eventos, hexagonal.
Patrón de organización	MVC, repositorio, inyección de dependencias, API Gateway, publicador-suscriptor.
Tecnología o framework	Angular, Ionic, NestJS, FastAPI, PostgreSQL, Docker, Terraform.

Idea clave

Angular o NestJS no son arquitecturas por sí mismos; son tecnologías que pueden usarse dentro de distintas arquitecturas.

1. Arquitectura cliente-servidor



Idea principal

- El cliente solicita servicios.
- El servidor procesa la solicitud.
- La respuesta vuelve al cliente.

Uso típico

- Aplicaciones web y móviles.
- API REST.
- Frontend separado del backend.

Cliente-servidor: ventajas y riesgos

Ventajas

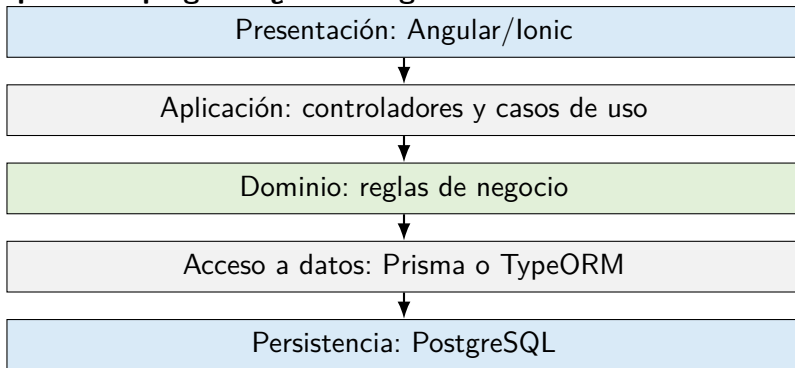
- Separa interfaz y procesamiento.
- Permite varios clientes.
- Centraliza reglas de negocio.
- Facilita reutilizar la API.

Riesgos

- Dependencia de la red.
- Posible punto único de fallo.
- Acoplamiento si la API está mal diseñada.
- Requiere seguridad en comunicación y acceso.

2. Arquitectura en capas

Responde la pregunta ¿Cómo organizo internamente el software?



Cada capa tiene una responsabilidad específica y reduce el acoplamiento entre interfaz, lógica y datos.

Arquitectura en capas: evaluación rápida

Fortalezas

- Mantenibilidad.
- Separación de responsabilidades.
- Pruebas por nivel.
- Cambios más controlados.

Debilidades

- Puede generar demasiados intermediarios.
- Algunas capas pueden quedar vacías.
- Una operación puede atravesar muchas capas.

Pregunta

¿En qué capa ubicaría la validación de permisos? ¿Y la validación visual de un formulario?

Ejemplo: mala separación de responsabilidades

```
// Componente de interfaz con demasiadas responsabilidades
async guardarProyecto() {
  if (!this.form.value.nombre) return;

  const total = this.form.value.horas * 15000;
  const respuesta = await fetch('/api/proyectos', {
    method: 'POST',
    body: JSON.stringify({ ...this.form.value, total })
  });

  if (respuesta.ok) {
    localStorage.setItem('ultimoProyecto', this.form.value.nombre);
  }
}
```

Preguntas para discutir

¿Qué corresponde a presentación, qué es regla del negocio y qué debería delegarse a un servicio?

Ejemplo: responsabilidades mejor separadas

```
// Componente Angular
crearProyecto(): void {
  if (this.form.invalid) return;
  this.proyectoService.crear(this.form.getRawValue()).subscribe();
}

// Servicio de aplicacion/backend
crearProyecto(dto: CrearProyectoDto) {
  const total = this.tarificador.calcular(dto.horas);
  return this.repositorio.guardar({ ...dto, total });
}
```

Resultado

La interfaz coordina la interacción; la regla de cálculo puede probarse sin navegador ni base de datos.

3. Arquitectura monolítica

Aplicación

- | - autenticación
- | - usuarios
- | - proyectos
- | - recomendaciones
- | - persistencia

Idea principal

- Todo se construye y despliega como una unidad.
- Es simple al inicio.
- Adecuada para equipos pequeños.

Advertencia

Un monolito no es malo por definición. El problema aparece cuando se vuelve desordenado, rígido y fuertemente acoplado.

4. Monolito modular

```
src/  
|- auth/  
|- users/  
|- projects/  
|- information/  
|- recommendations/
```

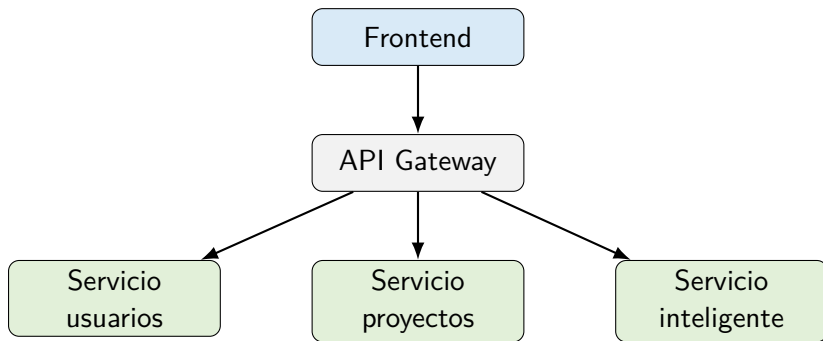
Idea principal

- Un solo despliegue.
- Módulos internos bien delimitados.
- Puede evolucionar hacia servicios separados.

Recomendación para el curso

Puede ser una arquitectura inicial adecuada para el backend NestJS del proyecto integrador.

5. Arquitectura de microservicios



Cada servicio puede evolucionar, escalar y desplegarse de manera independiente.

Microservicios: cuándo sí y cuándo no

Conviene cuando

- Hay equipos independientes.
- Existen dominios claros.
- Se requiere escalar servicios específicos.
- Hay madurez DevOps y observabilidad.

No conviene cuando

- El equipo es pequeño.
- El dominio aún no está claro.
- No hay automatización ni monitoreo.
- La complejidad supera el beneficio.

Idea principal

Organiza capacidades reutilizables como servicios: autenticación, búsqueda, notificaciones, recomendación o procesamiento de documentos.

Servicios posibles

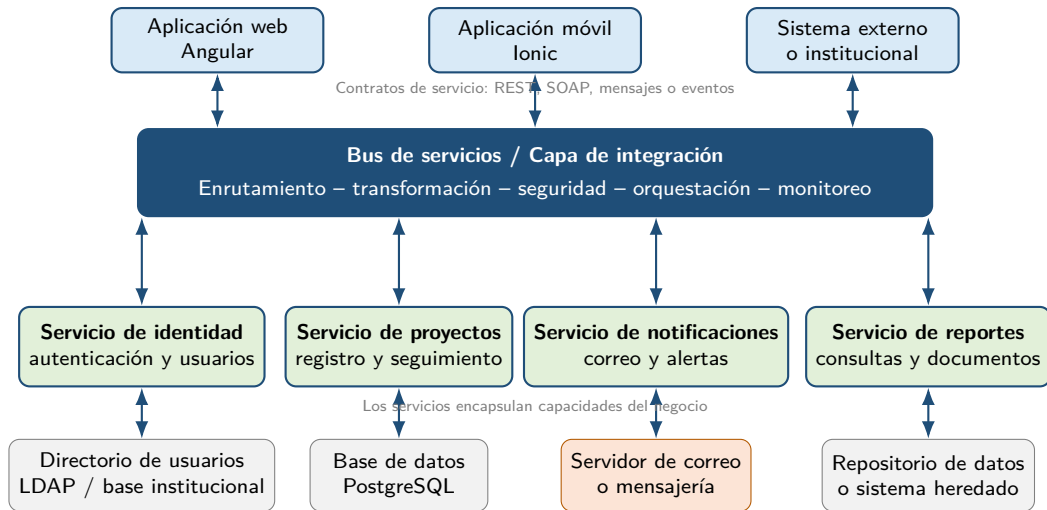
- Autenticación.
- Notificaciones.
- Búsqueda.
- Recomendaciones.
- Procesamiento de datos.

Diferencia con microservicios

- Los servicios pueden ser más grandes.
- Puede existir infraestructura compartida.
- Puede haber coordinación centralizada.

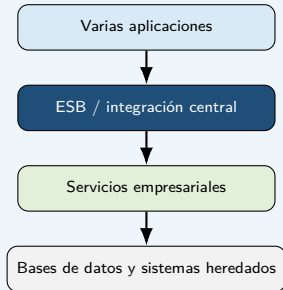
Arquitectura orientada a servicios

Service-Oriented Architecture – SOA



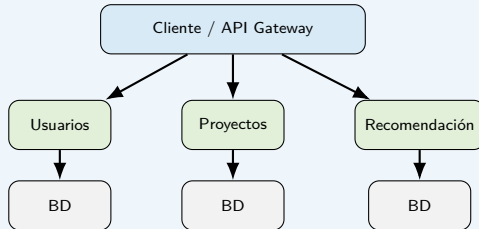
SOA frente a microservicios

SOA



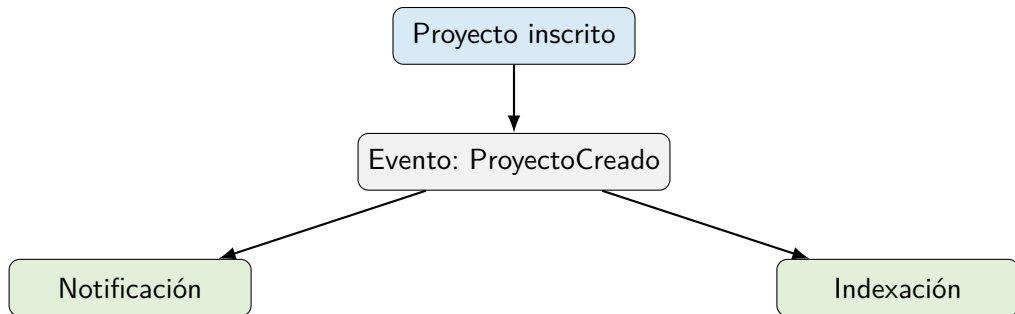
- Integración organizacional.
- Servicios de mayor alcance.
- Gobierno centralizado.
- Reutilización entre aplicaciones.

Microservicios



- Servicios autónomos.
- Despliegue independiente.
- Datos controlados por servicio.
- Gobierno descentralizado.

Arquitectura orientada a eventos



Los componentes no se llaman directamente; reaccionan a eventos publicados por otros componentes.

Eventos: ventajas y desafíos

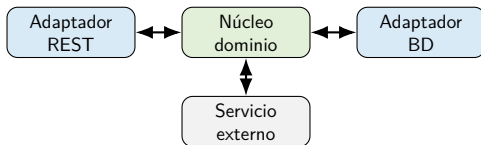
Ventajas

- Menor acoplamiento.
- Procesos asíncronos.
- Nuevos consumidores sin cambiar el origen.
- Útil para auditoría e indexación.

Desafíos

- Es más difícil seguir el flujo.
- Manejo de reintentos y errores.
- Eventos duplicados.
- Consistencia eventual.

Arquitectura hexagonal



También llamada: puertos y adaptadores.

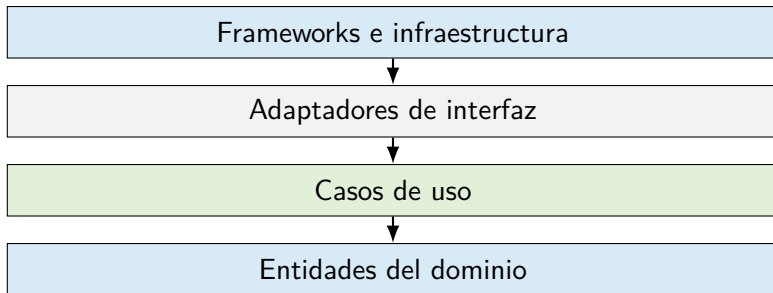
Elementos:

- núcleo del dominio;
- puertos o interfaces;
- adaptadores externos.

Busca:

- proteger el dominio;
- reducir dependencia de frameworks;
- facilitar pruebas;
- reemplazar adaptadores.

Arquitectura limpia



Regla central

Las dependencias deben apuntar hacia el núcleo del sistema, no hacia los detalles tecnológicos.

Arquitectura basada en componentes

AppComponent

- | - HeaderComponent
- | - ProjectListComponent
- | - ProjectFormComponent
- | - ProjectDetailComponent

Idea principal

- Divide la interfaz en componentes reutilizables.
- Cada componente tiene una responsabilidad clara.
- Favorece mantenibilidad y reutilización.

Aplicación

Es fundamental para Angular e Ionic.

Portal principal

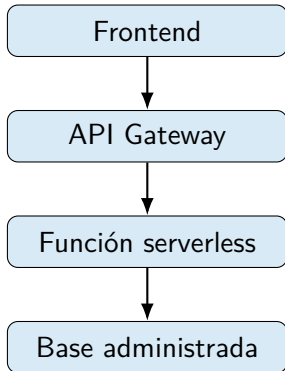
- |- frontend usuarios
- |- frontend proyectos
- |- frontend analítica

Uso recomendado

- Productos frontend muy grandes.
- Equipos independientes.
- Despliegues separados.

Cuidado

Suele ser innecesario en proyectos académicos pequeños por su complejidad de integración.



Ventajas

- Escalado automático.
- Menor administración de servidores.
- Pago asociado al uso.

Desafíos

- Dependencia del proveedor.
- Arranques en frío.
- Observabilidad más compleja.

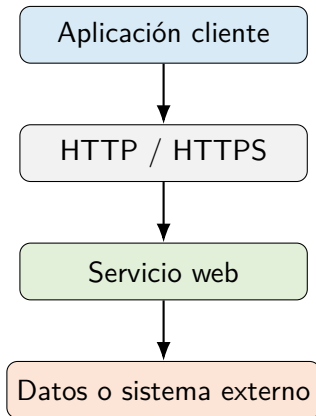
¿Qué es un servicio web?

Definición

Un **servicio web** es una capacidad de software accesible a través de una red, diseñada para permitir la comunicación entre aplicaciones mediante protocolos y formatos estandarizados.

Características habituales

- comunicación máquina a máquina;
- acceso mediante una red;
- contrato de servicio;
- intercambio de datos estructurados;
- independencia relativa entre cliente y servidor;
- reutilización por varias aplicaciones.



Tipos frecuentes de servicios web

SOAP

- protocolo formal;
- mensajes XML;
- contratos WSDL;
- estándares de seguridad y transacciones;
- frecuente en integración empresarial.

Servicios REST

- utilizan recursos identificados mediante URL;
- emplean métodos HTTP;
- normalmente intercambian JSON;
- son comunes en aplicaciones web y móviles;
- favorecen contratos simples y desacoplados.

Precisión

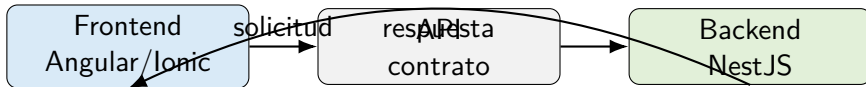
REST es un **estilo arquitectónico**; no es un protocolo. HTTP es el protocolo utilizado con mayor frecuencia para implementarlo.

¿Qué es una API?

Definición

Una **API** (*Application Programming Interface*) es un contrato que define cómo un componente de software puede solicitar operaciones, intercambiar datos o utilizar capacidades ofrecidas por otro componente.

- una API puede ser local o remota;
- no toda API es un servicio web;
- una **API web** expone operaciones mediante tecnologías web;
- una API define entradas, salidas, errores y reglas de uso.



Elementos de una API web

Elemento	Descripción	Ejemplo
Recurso	Entidad o información expuesta por la API.	Proyecto, usuario o documento.
Endpoint	Dirección que identifica una operación o recurso.	/api/proyectos
Método HTTP	Indica el tipo de operación solicitada.	GET, POST, PATCH, DELETE.
Representación	Formato utilizado para intercambiar información.	JSON o XML.
Código de estado	Informa el resultado de la solicitud.	200, 201, 404, 500.
Autenticación	Permite identificar al consumidor.	JWT, OAuth 2.0 o clave de API.
Contrato	Documenta operaciones, parámetros y respuestas.	OpenAPI/Swagger.

Ejemplo de API REST

Endpoints

```
GET    /api/proyectos
GET    /api/proyectos/15
POST   /api/proyectos
PATCH /api/proyectos/15
DELETE /api/proyectos/15
```

Ejemplo JSON

```
1 {
2   "nombre": "Aula Adaptativa",
3   "area": "Educacion",
4   "estado": "propuesto"
5 }
```

Flujo básico

El cliente envía una solicitud HTTP; el backend valida la información, ejecuta la lógica de negocio, accede a los datos y devuelve una respuesta.

¿Qué es una API?

Definición

Una **API** (*Application Programming Interface*) es un contrato que define cómo un componente de software puede solicitar operaciones, intercambiar datos o utilizar capacidades ofrecidas por otro componente.

- una API puede ser local o remota;
- no toda API es un servicio web;
- una **API web** expone operaciones mediante tecnologías web;
- una API define entradas, salidas, errores y reglas de uso.

Métodos HTTP y operaciones frecuentes

Método	Ejemplo	Intención	Respuesta típica
GET	/api/proyectos	Consultar una colección	200
GET	/api/proyectos/42	Consultar un recurso	200 o 404
POST	/api/proyectos	Crear un recurso	201
PUT	/api/proyectos/42	Reemplazar un recurso	200/204
PATCH	/api/proyectos/42	Modificar parcialmente	200/204
DELETE	/api/proyectos/42	Eliminar un recurso	204/404

Ejemplo de petición y respuesta

Petición

```
POST /api/proyectos HTTP/1.1
Content-Type: application/json
Authorization: Bearer <token>

{
  "nombre": "Buscador Web",
  "descripcion": "Proyecto integrador"
}
```

Respuesta

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/proyectos/42

{
  "id": 42,
  "nombre": "Buscador Web",
  "estado": "creado"
}
```

Códigos de estado: comunicar correctamente

Código	Significado	Ejemplo de uso
200	Operación exitosa	Consulta o actualización con cuerpo de respuesta.
201	Recurso creado	Creación exitosa mediante POST.
400	Solicitud inválida	Estructura o datos de entrada incorrectos.
401	No autenticado	Falta una credencial válida.
403	No autorizado	La identidad existe, pero no tiene permiso.
404	No encontrado	El recurso solicitado no existe.
409	Conflicto	Duplicidad o estado incompatible.
500	Error interno	Falla no controlada del servidor.

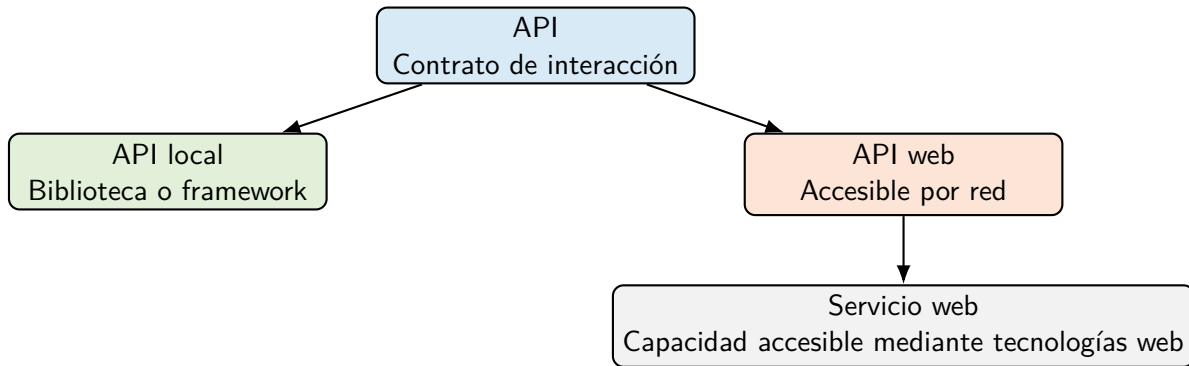
Buenas prácticas para diseñar endpoints

- Utilizar sustantivos para representar recursos: `/proyectos`, no `/crearProyecto`.
- Mantener nombres y convenciones consistentes.
- Validar datos de entrada en el servidor.
- No exponer detalles internos de errores o credenciales.
- Incorporar paginación, filtros y ordenamiento cuando corresponda.
- Versionar el contrato cuando exista un cambio incompatible.
- Documentar la API mediante OpenAPI/Swagger.

Discusión

¿Es suficiente que una API “funcione”? ¿Qué ocurre si no es consistente, segura o documentada?

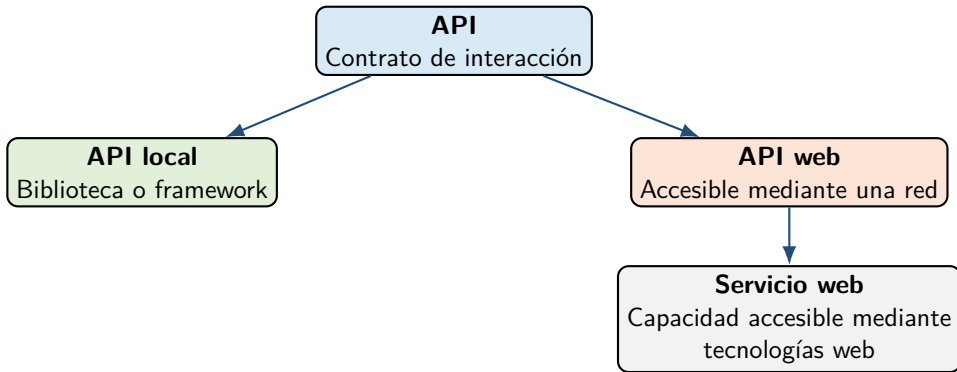
Relación entre servicio web y API



Idea clave

Una API define **cómo interactuar**; un servicio web proporciona **una capacidad accesible por red**. Una API web suele ser la interfaz mediante la cual se consume ese servicio.

Relación entre servicio web y API



Idea clave

Una API define **cómo interactuar**; un servicio web proporciona **una capacidad accesible mediante una red**. Una API web suele ser la interfaz mediante la cual una aplicación consume ese servicio.

La arquitectura se evalúa mediante atributos de calidad

Atributo	Pregunta orientadora	Evidencia posible
Mantenibilidad	¿Qué tan seguro y rápido es modificar el sistema?	Modularidad, pruebas, documentación.
Escalabilidad	¿Puede atender mayor carga sin degradarse de forma crítica?	Pruebas de carga, réplicas, caché.
Seguridad	¿Protege datos, identidad y operaciones?	Autenticación, autorización, validación.
Rendimiento	¿Responde dentro de límites aceptables?	Latencia, throughput, uso de recursos.
Observabilidad	¿Podemos inferir qué ocurre internamente?	Logs, métricas, trazas, health checks.

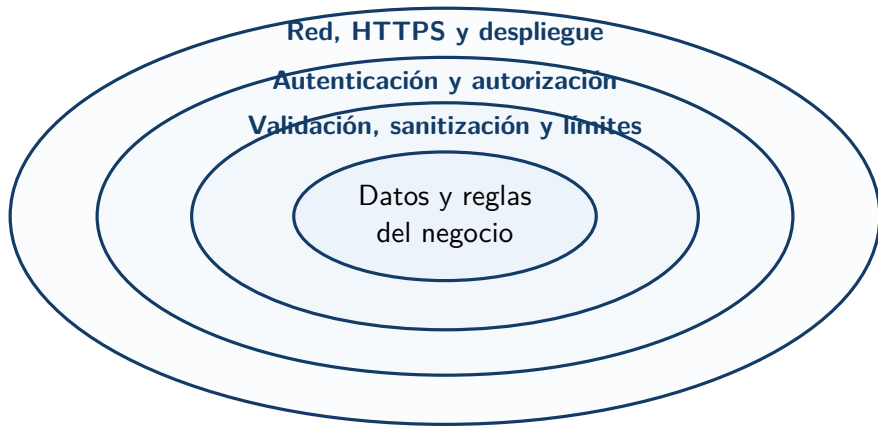
Señales de baja mantenibilidad

- lógica duplicada;
- funciones extensas;
- dependencias ocultas;
- nombres ambiguos;
- ausencia de pruebas;
- configuración dispersa;
- cambios pequeños con efectos impredecibles.

Decisiones favorables

- módulos cohesionados;
- contratos explícitos;
- separación de responsabilidades;
- revisión de código;
- automatización de pruebas;
- documentación de decisiones.

Seguridad: defensa en profundidad



Ningún control aislado reemplaza a los demás.

Observabilidad: no es lo mismo que monitoreo

Monitoreo

Permite vigilar condiciones conocidas mediante indicadores y alertas.

Observabilidad

Permite comprender estados internos a partir de señales externas, incluso ante fallas no previstas.

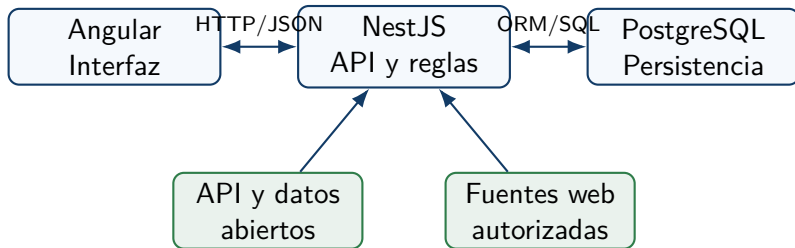
- **Logs:** eventos con contexto y correlación.
- **Métricas:** valores agregados a través del tiempo.
- **Trazas:** recorrido de una solicitud entre componentes.
- **Health checks:** disponibilidad y dependencias críticas.

Problema

Construir una aplicación que recopile recursos desde fuentes web autorizadas, permita buscarlos y recomiende contenidos según el perfil y la interacción del usuario.

- Frontend Angular para búsqueda, visualización y preferencias.
- API NestJS para reglas, seguridad e integración.
- PostgreSQL para usuarios, recursos, metadatos e interacciones.
- Procesos posteriores de API/scraping, indexación y recomendación.
- Pipeline CI/CD, Docker, despliegue y observabilidad.

Arquitectura inicial del caso guía



Actividad en equipos: 12 minutos

Añada autenticación, observabilidad y un mecanismo adaptativo. Justifique dónde ubicaría cada componente.

Matriz de decisiones arquitectónicas

Decisión	Alternativas	Criterio para justificar
Comunicación frontend–backend	REST, GraphQL	Complejidad, caché, claridad del contrato.
Acceso a datos	Prisma, TypeORM, SQL directo	Productividad, control y mantenibilidad.
Autenticación	Sesión, JWT	Arquitectura, riesgo y ciclo de vida.
Procesamiento web	Síncrono, tarea en segundo plano	Latencia, volumen y tolerancia a fallas.
Recomendación	Reglas, contenido, híbrida	Datos disponibles, transparencia y utilidad.
Despliegue	Monolito modular, servicios separados	Escala, costo y complejidad operativa.

Frontend

Angular + TypeScript

- componentes;
- router;
- servicios;
- formularios reactivos;
- RxJS y signals.

Backend

NestJS + TypeScript

- módulos;
- controladores;
- servicios;
- DTO y validación;
- Swagger.

Persistencia

PostgreSQL

- modelos;
- relaciones;
- migraciones;
- consultas;
- integridad.

Estructura inicial sugerida del repositorio

Monorepositorio para comenzar

proyecto-integrador/

| - frontend/

Angular

| - backend/

NestJS

| - database/

scripts y migraciones

| - docs/

arquitectura y decisiones

| - infra/

Docker/Terraform

| - .github/workflows/

CI/CD

| - docker-compose.yml

| - README.md

Principio

La estructura debe anticipar pruebas, despliegue, documentación e infraestructura; no solo el código funcional.

Acuerdos mínimos de trabajo colaborativo

- El repositorio principal debe permanecer ejecutable.
- Los cambios se integran mediante ramas y pull requests.
- Cada pull request debe explicar propósito, pruebas y efectos.
- No se incorporan credenciales ni secretos al repositorio.
- Las decisiones relevantes se registran en documentación de arquitectura.
- Cada cambio debe vincularse con una tarea o historia de trabajo.
- La revisión de código evalúa corrección, claridad, seguridad y pruebas.

Actividad práctica del segundo bloque

- ➊ Crear o confirmar el equipo de proyecto.
- ➋ Definir un nombre y una necesidad inicial.
- ➌ Crear el repositorio y el archivo README.md.
- ➍ Incorporar carpetas frontend, backend, docs e infra.
- ➎ Dibujar la arquitectura inicial en docs/arquitectura.md.
- ➏ Registrar tres decisiones y dos riesgos técnicos.
- ➐ Realizar un primer commit con un mensaje significativo.

Producto de salida

Repositorio accesible, estructura inicial y diagrama arquitectónico justificable.

Síntesis: cinco ideas que deben quedar claras

- ❶ Cliente, servidor y base de datos poseen responsabilidades distintas.
- ❷ Una arquitectura en capas controla dependencias y facilita cambios y pruebas.
- ❸ Una API REST es un contrato, no solo una colección de URLs.
- ❹ La calidad debe expresarse mediante atributos y evidencia medible.
- ❺ El proyecto comienza con arquitectura, repositorio y acuerdos, no únicamente con código.

Ticket de salida

Responder individualmente en 5 minutos:

- 1 Dibuje el flujo de una solicitud desde Angular hasta PostgreSQL y de regreso.
- 2 Explique una decisión que mejore mantenibilidad.
- 3 Indique un control de seguridad que deba existir en el backend.
- 4 Proponga una señal de observabilidad para una API.
- 5 Escriba una duda que todavía necesite aclaración.

Uso docente

Las respuestas permiten ajustar la profundidad de las clases siguientes sobre DevOps, Git e integración continua.

Trabajo autónomo para la próxima clase

- Revisar el flujo completo del proyecto y sus responsabilidades.
- Verificar instalación de Git, Node.js, Docker y editor de código.
- Leer el README.md del repositorio del curso.
- Proponer una convención de ramas y mensajes de commit.
- Identificar un riesgo de seguridad y uno de mantenibilidad del proyecto.

Próxima clase: ciclo de vida, DevOps, DevSecOps y trabajo colaborativo con Git.

Pauta breve para retroalimentar el diagnóstico

Aspecto	Evidencia de comprensión inicial
Cliente–servidor	Distingue interacción, procesamiento y persistencia.
Capas	Separa presentación, aplicación, dominio e infraestructura.
REST/HTTP	Relaciona recurso, método, código y representación.
Calidad	Justifica decisiones mediante atributos verificables.
Seguridad	Reconoce que los controles críticos pertenecen al servidor.
Observabilidad	Propone logs, métricas, trazas o health checks con propósito.

Lista de verificación del producto inicial

- ☐ Problema y usuario objetivo definidos.
- ☐ Responsabilidades de frontend, backend y persistencia descritas.
- ☐ Diagrama de arquitectura comprensible.
- ☐ Repositorio creado con estructura inicial.
- ☐ README con instrucciones mínimas.
- ☐ Tres decisiones arquitectónicas registradas.
- ☐ Dos riesgos técnicos identificados.
- ☐ Primer commit realizado sin secretos ni archivos innecesarios.